Part I: Divide and Conquer

Lecture 3: The Polynomial Multiplication Problem

Computer Science and Technology
United International College

Outline

Divide-and-Conquer

Objective

- Show a second example of divide-and-conquer

Divide

- Divide a given problem into two or more subproblems (ideally of approximately equal size).

Conquer

- Solve each subproblem (directly if small enough or recursively).

Combine

- Combine the solutions of the subproblems into a global solution.

Outline

The Polynomial Multiplication Problem

Definition (Polynomial Multiplication Problem)

Given two polynomials

$$A(x) = a_0 + a_1 x + \ldots + a_n x^n$$
$$B(x) = b_0 + b_1 x + \ldots + b_m x^m$$

compute the product $A(x)B(x)$.

Examples

$$A(x) = 1 + 2x + 3x^2, B(x) = 3 + 2x + 2x^2$$
$$A(x)B(x) = 3 + 8x + 15x^2 + 10x^3 + 6x^4$$

- Assume that the coefficients a_i and b_i are stored in arrays $A[0 \cdots n]$ and $B[0 \cdots m]$.
- The cost is the number of scalar multiplications and additions.

What do we need to computer exactly?

Let

- $A(x) = \sum_{i=0}^{n} a_i x^i$
- $B(x) = \sum_{i=0}^{m} b_i x^i$
- $C(x) = A(x)B(x) = \sum_{k=0}^{n+m} c_k x^k$

Then,

$$c_k = \sum_{0 \leq i \leq n, 0 \leq j \leq m, i+j=k} a_i b_j, \text{ for all } 0 \leq k \leq m+n$$

Definition

The vector $(c_0, c_1, ..., c_{m+n})$ is the convolution of the vectors $(a_0, a_1, ..., a_n)$ and $(b_0, b_1, ..., b_m)$.

We need to calculate convolutions. This is a major problem in digital signal processing.

Outline

Direct (Brute Force) Approach

To ease analysis, assume n = m.

- $A(x) = \sum_{i=0}^{n} a_i x^i$
- $B(x) = \sum_{i=0}^{n} b_i x^i$
- $C(x) = A(x)B(x) = \sum_{k=0}^{2n} c_k x^k$ with

$$c_k = \sum_{0 \le i,j \le n, i+j=k} a_i b_j, \text{ for all } 0 \le k \le 2n$$

Direct approach: Compute all c_k's using the formula above

- Total number of multiplications: $\Theta(n^2)$
- Total number of additions: $\Theta(n^2)$
- Complexity: $\Theta(n^2)$

Outline

Divide-and-Conquer: Divide I

Let

- $A_0(x) = a_0 + a_1 x + \ldots + a_{\lceil \frac{n}{2} \rceil - 1} x^{\lceil \frac{n}{2} \rceil - 1}$
- $A_1(x) = a_{\lceil \frac{n}{2} \rceil} + a_{\lceil \frac{n}{2} \rceil + 1} x + \ldots + a_n x^{n - \lceil \frac{n}{2} \rceil}$
- $A(x) = A_0(x) + A_1(x) x^{\lceil \frac{n}{2} \rceil}$

Similarly, we define $B_0(x)$ and $B_1(x)$ such that

- $B(x) = B_0(x) + B_1(x) x^{\lceil \frac{n}{2} \rceil}$

$$A(x)B(x) = A_0(x)B_0(x) + A_0(x)B_1(x)x^{\lceil \frac{n}{2} \rceil} + A_1(x)B_0(x)x^{\lceil \frac{n}{2} \rceil} + A_1(x)B_1(x)x^{2\lceil \frac{n}{2} \rceil}$$

The original problem (of size n) is divided into 4 problems of input size $\frac{n}{2}$.

Divide-and-Conquer: Divide II

Examples

$$A(x) = 2 + 5x + 3x^2 + x^3 - x^4$$

$$B(x) = 1 + 2x + 2x^2 + 3x^3 + 6x^4$$

$$A(x)B(x) = 2 + 9x + 17x^2 + 23x^3 + 34x^4 + 39x^5$$

$$+19x^6 + 3x^7 - 6x^8$$

$$A_0(x) = 2 + 5x, A_1(x) = 3 + x - x^2, A(x) = A_0(x) + A_1(x)x^2$$

$$B_0(x) = 1 + 2x, B_1(x) = 2 + 3x + 6x^2, B(x) = B_0(x) + B_1(x)x^2$$

$$A_0(x)B_0(x) = 2 + 9x + 10x^2$$

$$A_1(x)B_1(x) = 6 + 11x + 19x^2 + 3x^3 - 6x^4$$

$$A_0(x)B_1(x) = 4 + 16x + 27x^2 + 30x^3$$

$$A_1(x)B_0(x) = 3 + 7x + x^2 - 2x^3$$

$$A_0(x)B_1(x) + A_1(x)B_0(x) = 7 + 23x + 28x^2 + 28x^3$$

$$A_0(x)B_0(x) + (A_0(x)B_1(x) + A_1(x)B_0(x))x^2 + A_1(x)B_1(x)x^4$$

$$= 2 + 9x + 17x^2 + 23x^3 + 34x^4 + 39x^5 + 19x^6 + 3x^7 - 6x^8$$

Divide-and-Conquer: Conquer

Conquer: Solve the four subproblems

- compute

$$A_0(x)B_0(x), A_0(x)B_1(x), A_1(x)B_0(x), A_1(x)B_1(x)$$

  by recursively calling the algorithm 4 times.

Combine

- add the following four polynomials

$$A(x)B(x) = A_0(x)B_0(x) + A_1(x)B_1(x)x^{\lceil \frac{n}{2} \rceil} + A_1(x)B_0(x)x^{\lceil \frac{n}{2} \rceil} + A_1(x)B_1(x)x^{2\lceil \frac{n}{2} \rceil}$$

- takes $\Theta(n)$ operations (Why?)

The First Divide-and-Conquer Algorithm

Algorithm 1 PolyMulti1(A(x), B(x))

Input: Two polynomials $A(x)$ and $B(x)$ of order n

Output: The polynomial $C(x) = A(x)B(x)$

1: **if** n=0 **then**
2: **return** $a_0 \times b_0$
3: **else**
4: $A_0(x) = a_0 + a_1x + ... + a_{\lceil \frac{n}{2} \rceil - 1}x^{\lceil \frac{n}{2} \rceil - 1}$
5: $A_1(x) = a_{\lceil \frac{n}{2} \rceil} + a_{\lceil \frac{n}{2} \rceil + 1}x + ... + a_n x^{n - \lceil \frac{n}{2} \rceil}$
6: $B_0(x) = b_0 + b_1x + ... + b_{\lceil \frac{n}{2} \rceil - 1}x^{\lceil \frac{n}{2} \rceil - 1}$
7: $B_1(x) = b_{\lceil \frac{n}{2} \rceil} + b_{\lceil \frac{n}{2} \rceil + 1}x + ... + b_n x^{n - \lceil \frac{n}{2} \rceil}$
8: $U(x) = PolyMulti1(A_0(x), B_0(x))$
9: $V(x) = PolyMulti1(A_0(x), B_1(x))$
10: $W(x) = PolyMulti1(A_1(x), B_0(x))$
11: $Z(x) = PolyMulti1(A_1(x), B_1(x))$
12: **return** $U(x) + (V(x) + W(x))x^{\lceil \frac{n}{2} \rceil} + Z(x)x^{2\lceil \frac{n}{2} \rceil}$
13: **end if**

Analysis of Running Time

$$
\begin{aligned}
T(n) &= 4(4T(\frac{n}{2^2}) + c\frac{n}{2}) + cn \\
&= 4^2 T(\frac{n}{2^2}) + (1+2)cn \\
&= 4^3 T(\frac{n}{2^3}) + (1+2+2^2)cn \\
&\vdots \\
&= 4^i T(\frac{n}{2^i}) + \sum_{j=0}^{i-1} 2^j cn \\
&= (2^i)^2 T(\frac{n}{2^i}) + cn(2^i - 1)
\end{aligned}
$$

Let $n = 2^i$,

$$
\begin{aligned}
T(n) &= n^2 T(1) + cn(n-1) \\
&= \Theta(n^2)
\end{aligned}
$$

Same order as the brute force approach! No improvement!

Outline

Two Observations

Observation

What we really need are the following 3 terms.

$$A_0B_0, A_0B_1 + A_1B_0, A_1B_1?$$

Instead of the following 4 terms.

$$A_0B_0, A_0B_1, A_1B_0, A_1B_1?$$

Observation

The three terms can be obtained using only 3 multiplications.

$$Y = (A_0 + A_1)(B_0 + B_1)$$
$$U = A_0B_0$$
$$Z = A_1B_1$$

- U and Z are what we originally wanted.
- $A_0B_1 + A_1B_0 = Y - U - Z$

The Second Divide-and-Conquer Algorithm

Algorithm 2 $PolyMulti2(A(x), B(x))$

Input: Two polynomials $A(x)$ and $B(x)$ of order n

Output: The polynomial $C(x) = A(x)B(x)$

1: **if** n=0 **then**
2: **return** $a_0 \times b_0$
3: **else**
4: $A_0(x) = a_0 + a_1x + ... + a_{\lceil \frac{n}{2} \rceil - 1}x^{\lceil \frac{n}{2} \rceil - 1}$
5: $A_1(x) = a_{\lceil \frac{n}{2} \rceil} + a_{\lceil \frac{n}{2} \rceil + 1}x + ... + a_n x^{n - \lceil \frac{n}{2} \rceil}$
6: $B_0(x) = b_0 + b_1x + ... + b_{\lceil \frac{n}{2} \rceil - 1}x^{\lceil \frac{n}{2} \rceil - 1}$
7: $B_1(x) = b_{\lceil \frac{n}{2} \rceil} + b_{\lceil \frac{n}{2} \rceil + 1}x + ... + b_n x^{n - \lceil \frac{n}{2} \rceil}$
8: $Y(x) = PolyMulti2(A_0(x) + A_1(x), B_0(x) + B_1(x))$
9: $U(x) = PolyMulti2(A_0(x), B_0(x))$
10: $Z(x) = PolyMulti2(A_1(x), B_1(x))$
11: **return** $U(x) + (Y(x) - U(x) - Z(x))x^{\lceil \frac{n}{2} \rceil} + Z(x)x^{2\lceil \frac{n}{2} \rceil}$
12: **end if**

Running Time of the Modified Algorithm

$$
\begin{aligned}
T(n) &= 3(3T(\frac{n}{2^2}) + c\frac{n}{2}) + cn \\
&= 3^2 T(\frac{n}{2^2}) + (1 + \frac{3}{2})cn \\
&= 3^3 T(\frac{n}{2^3}) + (1 + \frac{3}{2} + (\frac{3}{2})^2)cn \\
&\vdots \\
&= 3^i T(\frac{n}{2^i}) + \sum_{j=0}^{i-1} (\frac{3}{2})^i cn \\
&= 3^i T(\frac{n}{2^i}) + 2cn((\frac{3}{2})^i - 1)
\end{aligned}
$$

Running Time of the Modified Algorithm

Let $i = \log_2 n$, then

$$T(n) = 3^{\log_2 n} T(1) + 2cn(\frac{3}{2})^{\log_2 n} - 2cn$$

By changing the base of logarithm,

- $3^{\log_2 n} = 3^{(\log_3 n)(\log_2 3)} = n^{\log_2 3}$
- $(\frac{3}{2})^{\log_2 n} = (\frac{3}{2})^{(\log_{\frac{3}{2}} n)(\log_2 \frac{3}{2})} = n^{\log_2 \frac{3}{2}} = n^{(\log_2 3)-1}$

Thus,

$$\begin{aligned} T(n) &= n^{\log_2 3} + 2cn^{\log_2 3} - 2cn \\ &= \Theta(n^{\log_2 3}) \end{aligned}$$

- The second method is much better!

Outline

Remarks

- The divide-and-conquer approach does not always give you the best solution.
  - Our original algorithm was just as bad as brute force.
- There is actually an $\mathcal{O}(nlogn)$ solution to the polynomial multiplication problem.
  - It involves using the Fast Fourier Transform algorithm as a subroutine.
  - The FFT is another classic divide-and-conquer algorithm (Chapt 30 in CLRS gives details).
- The idea of using 3 multiplications instead of 4 is used in large-integer multiplications.
  - A similar idea is the basis of the classic Strassen matrix multiplication algorithm (CLRS, Chapter 28).